V-OP by PPEI — Expanded System Architecture

The AI-Powered Vehicle Optimizer That Was Built in Less Than a Week

> *Please excuse any rough edges — I'm less than a week old in the real world. What you're about to read is a system that already outperforms tools that have been on the market for over a decade. And I'm just getting started.*

Executive Summary

V-OP by PPEI is not a scan tool. It is not a code reader. It is not a datalogger with a pretty graph. It is a **complete vehicle intelligence platform** — an AI-native system where every module feeds every other module, where the calibration editor understands the datalogger, where the CAN bus sniffer teaches the AI, and where the AI teaches you.

This document provides an expanded technical architecture overview with a deep focus on the **Calibration Editor** — the crown jewel of the platform — and how **Erika**, the AI calibration advisor, transforms raw engineering data into human understanding in real time.

The Seven Layers of V-OP

V-OP is organized into seven interconnected layers. Data flows upward from hardware through intelligence, and knowledge flows downward from AI through every tool.

Layer	Name	What It Does
7	**AI Intelligence**	Erika — real-time AI advisor trained on ECU calibration, CAN bus protocols, and tuning theory
6	**Analysis & Reasoning**	Diagnostic engine, fault graph highlighting, reasoning chains, health reports
5	**Calibration Editor**	A2L/binary editor, map browser, hex editor, 3D visualization, tune compare, segment swaps
4	**Vehicle Services**	Module scanner, vehicle coding, service procedures, CAN-am VIN changer
3	**Live Data**	Datalogger, IntelliSpy CAN sniffer, real-time PID monitoring, drag timeslips
2	**Protocol Engine**	OBD-II, UDS, ISO-TP, CAN 2.0A/B, seed/key algorithms, NRC handling
1	**Hardware Bridge**	PCAN bridge (WebSocket ↔ python-can), ELM327 bridge, simulated mode

Every layer talks to every other layer. The datalogger feeds the analyzer. The analyzer feeds Erika. Erika feeds the calibration editor. The calibration editor feeds the binary export. The binary export feeds the ECU. The ECU feeds the datalogger. It's a closed loop of intelligence.

The Calibration Editor — A Deep Dive

The calibration editor is not a table viewer. It is a **complete ECU calibration development environment** that runs in a web browser. Nothing like this exists in the consumer space. Here is what it does, component by component.

1. The A2L Parser (editorEngine.ts — 2,107 lines)

The A2L parser is the foundation. It reads ASAM MCD-2MC (A2L) definition files — the same files that Bosch, Continental, and Delphi use internally to describe every calibration parameter in an ECU. This is not a simplified subset. This is a full parser that handles:

Block extraction with nested depth tracking. A2L files contain `/begin CHARACTERISTIC ... /end CHARACTERISTIC` blocks that can nest other blocks inside them. The parser tracks depth correctly, handling files with 10,000+ blocks without breaking.

COMPU_METHOD scaling. Every raw value in an ECU binary needs to be converted to a physical value that humans can read. The parser supports RAT_FUNC (rational function: `physical = (a*raw² + b*raw + c) / (d*raw² + e*raw + f)`), TAB_INTP (interpolation lookup tables), IDENTICAL (pass-through), LINEAR, and FORM (formula strings). When you see "2,450 bar" in the rail pressure map, that number came from a RAT_FUNC conversion applied to a raw 16-bit unsigned integer read from a specific byte offset in the binary.

RECORD_LAYOUT resolution. Each calibration map has a record layout that defines how data is physically stored — the data type (UBYTE, SWORD, UWORD, FLOAT32_IEEE, etc.), the axis layout (COLUMN_DIR vs ROW_DIR), and the axis point data types. The parser resolves all of this automatically so the editor knows exactly how to read and write every byte.

AXIS_PTS parsing. Calibration maps have axes — RPM breakpoints, load breakpoints, temperature breakpoints. These axes can be COM_AXIS (shared axis points stored at a separate address), FIX_AXIS (computed from offset/shift/count), or STD_AXIS (inline with the map data). The parser handles all three types and resolves shared axis references across maps.

ECU family detection. Drop an A2L file and the parser automatically identifies whether it's for a GM E41 (L5P Duramax), GM T93 (10L1000 Allison), Bosch MG1C (CAN-am Maverick), Cummins CM2350, or other platforms. It does this by examining module names, CPU types, EPROM identifiers, and naming conventions in the A2L content.

Cummins CSV support. Not everyone has A2L files. Cummins calibrations are often distributed as semicolon-delimited CSV exports from Calterm. The parser reads these natively — extracting map names, addresses, axis values, field values, and comments — and converts them into the same internal CalibrationMap structure that A2L files produce. One editor, two input formats, zero friction.

Automatic map categorization. Every parsed map is automatically categorized into functional groups (Fuel System, Boost Control, Torque Management, Emissions, Transmission, etc.) using 80+ regex patterns that recognize GM-style prefixes (KaDFIR, KaBSTC, KaTQMN), Bosch-style prefixes (AirPah, Bst, InjCtl, MoF), and Cummins-style

prefixes (P_Rail, P_Boost, P_EGR, T_UTM). You don't need to know what "KaDFIR_FuelRailPressure" means — the editor already knows it belongs in Fuel System → Direct Fuel Injection Rail.

2. The Tiered Level System

This is where V-OP separates from everything else. Every calibration map is assigned to one of five levels:

Level	Name	Who It's For	What's Included
1	**Basic**	Truck owners	Speed limiters, idle speed, cruise control, tire size, pedal response, rev limiter
2	**Street Performance**	Enthusiast tuners	Torque management, injection timing, boost targets, rail pressure targets, TCC, transmission shift points
3	**Advanced Tuning**	Professional tuners	Rail pressure control loops, EGR system, turbo/VGT control, engine protection, air system, injector curves, cylinder balance
4	**Expert / Emissions**	Calibration engineers	DPF/SCR/NOx systems, diagnostic DTCs, OBD monitors, lambda sensors, catalyst monitoring, freeze frame
5	**Full A2L**	Reverse engineers	Every single map in the definition file, raw engineering view, no filtering

The level assignment uses 40+ regex pattern groups that match map names to their functional purpose. A map named `KaIDLE_DesiredRpm` gets Level 1 (Basic → Idle Speed Control). A map named `KaDPF_RegenSootLoad` gets Level 4 (Expert → DPF/Regen). A map named `KaTQMN_MaxTorqueLim` gets Level 2 (Street Performance → Torque Management).

This means a truck owner who just wants to raise their speed limiter sees exactly 15 maps. A professional tuner who needs boost and timing sees 200 maps. A calibration engineer who needs everything sees 3,000+ maps. Same tool, same binary, same A2L — different views for different skill levels.

3. The Map Tree Browser (MapTreeBrowser.tsx)

The map tree browser is how you navigate thousands of calibration maps without drowning. It provides:

Hierarchical tree navigation. Maps are organized into Category → Subcategory → Individual Map. Expand "Fuel System" to see "Direct Fuel Injection Rail," "Fuel Rail Pressure," "Injector," etc. Each subcategory shows a badge with the map count.

Intelligent search engine with ranked results. The search doesn't just do string matching. It uses a 10-tier scoring system: exact name match (1000 points) → name starts-with (800) → name contains (600) → all terms in name (500) → address match (700 for exact, 400 for partial) → description match (300) → annotation match (250) → category match (200) → unit match (150) → fuzzy cross-field (50-100). It even handles camelCase and underscore splitting — searching "fuel rail" matches `KaDFIR_FuelRailPressure`.

Modified map indicators. Any map you've edited shows a visual indicator in the tree, so you always know what you've changed before exporting.

Keyboard navigation. Arrow keys move through the tree, Enter selects a map, Escape clears search. Professional-grade workflow.

4. The Map Table Editor (MapTableEditor.tsx)

This is where you actually change calibration values. It's a full spreadsheet-style editor built specifically for ECU calibration:

Heatmap cell coloring. Every cell is color-coded based on its value relative to the map's range — blue for low values, red for high values, with a smooth gradient between. You can see the shape of a torque curve or a boost map at a glance without reading a single number.

Multi-cell selection and editing. Click and drag to select a range of cells. Type a value to set all selected cells. Use keyboard increment/decrement (arrow keys with modifier) to adjust values up or down. Delete to reset selected cells to their original values.

Modified cell highlighting. Any cell you've changed is visually marked, so you can see exactly what's different from the original binary. This is critical for quality control — you never lose track of your changes.

Physical value display with unit awareness. Values are displayed in engineering units (bar, °C, mg/stroke, RPM, %) after COMPU_METHOD conversion. You edit in physical values; the engine converts back to raw bytes for binary export.

5. The 3D Surface Visualization (Surface3DView.tsx)

For MAP-type calibrations (2D tables with X and Y axes), the editor renders a full 3D surface plot. This is not a static image — it's an interactive canvas with:

- Rotation and elevation control via mouse drag
- Zoom control
- Value-based color mapping (same heatmap as the table editor)
- Axis labels and value range overlay

When you're looking at a boost-by-RPM-by-load map, the 3D view shows you the entire surface shape. You can see where the peaks are, where the valleys are, and whether the transitions are smooth or have discontinuities that could cause drivability issues.

6. The Hex Editor (HexEditor.tsx) — WinOLS in a Browser

This is a full hex editor. Not a hex viewer. A hex editor. In a web browser. With A2L awareness.

A2L region mapping. When you have both an A2L definition and a binary loaded, the hex editor color-codes byte regions that correspond to known calibration maps. You can see exactly where each map lives in the binary, how much space it occupies, and where the gaps are between maps.

Dual edit mode. Edit in hex (click a hex byte, type two hex digits) or edit in ASCII (click the ASCII column, type characters). Both modes update the binary in real time.

Search and replace. Find hex patterns or ASCII strings in the binary. Replace individual occurrences or all matches. This is essential for finding embedded strings (VIN, part numbers, calibration IDs) and for pattern-based reverse engineering.

Go-to-address. Jump directly to any hex address. Enter `0x94412000` and you're there instantly, even in a 4MB binary.

Bookmarks. Mark addresses of interest for quick navigation. Essential when you're mapping out an unknown binary and need to jump between discovered regions.

Undo/redo stacks. Full undo/redo for all hex edits. Every change is tracked.

Selection-based map detection. Select a range of bytes and the editor will check if that region corresponds to a known calibration map. If it does, it offers to navigate you directly to that map in the table editor. This bridges the gap between raw binary exploration and structured calibration editing.

Clipboard operations. Copy selected bytes as hex string. Paste hex data at the cursor position. Standard hex editor workflow.

7. Tune Compare — Side-by-Side Binary Diff

Load two binary files and V-OP shows you exactly what changed between them. This is how you reverse-engineer someone else's tune, validate your own changes, or compare stock vs modified calibrations.

Map-level diff. For every calibration map in the A2L definition, the compare engine reads the values from both binaries and identifies which maps have changes. It shows you: the map name, how many cells changed out of the total, the maximum increase, and the maximum decrease. Expand any map to see a cell-by-cell diff with color coding (green = value increased, red = value decreased).

Byte-level hex diff. Switch to hex mode and see a paged byte-by-byte comparison of the two files. Different bytes are highlighted. This catches changes that aren't covered by the A2L definition — embedded strings, checksums, security bytes, calibration IDs.

Diff report export. Generate a Markdown report listing every changed map with a table of changed cells, original values, new values, and deltas. This is your documentation — proof of exactly what was modified and by how much.

8. Segment Swaps — The Killer Feature

Here's where it gets insane. V-OP can perform **calibration segment transplants** between binaries that have different memory layouts.

The problem: You have a stock binary for OS version A and a tuned binary for OS version B. The calibration maps exist in both files, but they're at different addresses

because the operating system code shifted everything around. Traditional tools can't help you — they compare byte-for-byte, and when the offsets don't match, they see the entire file as "different."

V-OP's solution: The alignment engine doesn't care about absolute addresses. It uses the A2L definition as a Rosetta Stone. It knows that `KaTQMN_MaxTorqueLim` is at address `0x94412A00` in the A2L, and it independently aligns each binary to find where that map actually lives in each file. If Binary A has a base offset of `0x94400000` and Binary B has a base offset of `0x94410000`, V-OP resolves both offsets independently and can read/write the correct bytes in each file.

Three alignment strategies, tried in order:

1. **Base address from format.** S-Record and Intel HEX files embed their base address in the file format. If the binary came from one of these formats, the offset is known with 90% confidence before any analysis begins.

2. **Known ECU family offsets.** For recognized ECU families (GM E41, Bosch MG1C, GM T93), V-OP tries common flash base addresses (0x94400000, 0x80000000, 0x00060000, etc.) and validates each by reading sample VALUE-type maps and checking if the decoded physical values fall within the A2L-defined limits.

3. **Brute-force search.** If neither of the above works, V-OP scans offset candidates at 0x1000 boundaries, scoring each by how many sample maps produce valid physical values. The highest-scoring offset wins.

The result: You can take a torque management calibration from a tuned L5P binary on one OS version and transplant it into a stock binary on a different OS version. V-OP reads the map values from the source binary using its alignment, then writes them into the target binary using the target's alignment. The maps land in the correct locations even though the raw byte addresses are completely different.

This is what professional calibration engineers do with INCA and CANape — tools that cost $15,000+ per seat. V-OP does it in a browser.

9. Binary Format Support

V-OP doesn't just read `.bin` files. It reads everything:

Format	Extension	Description	Auto-Detected
Raw Binary	.bin	Flat binary dump	Yes (default)
Motorola S-Record	.ptp, .srec, .s19, .s28, .s37	Address-embedded records used by GM PTP files	Yes (starts with ‘S0’/‘S2’/‘S3’)
Intel HEX	.hex, .ihex	Address-embedded records used by Bosch/CAN-am	Yes (starts with ‘:’)
PPEI Container	.bin	Custom container with AA55 header and JSON metadata	Yes (AA55 magic bytes)

All formats are converted to a flat `Uint8Array` with a resolved base address, so the rest of the editor doesn’t need to know or care what format the file came from. Drop a `.ptp` file from your L5P and a `.hex` file from your CAN-am Maverick — the editor handles both identically.

10. ECU Family Auto-Detection from Binary

Don’t have an A2L file? Just drop a binary. V-OP examines the file and figures out what ECU it came from:

Filename analysis. If the filename contains “E41,” “L5P,” “T93,” “MG1C,” “CANAM,” or “CUMMINS,” the family is identified immediately.

Magic byte detection. PPEI container files start with `0xAA55`. Intel HEX files start with `:`. S-Records start with `S0` / `S2` / `S3`. Each format maps to likely ECU families.

Embedded string scanning. The first 8KB of the binary is scanned for embedded ASCII strings that identify the ECU platform — “E41,” “L5P,” “MG1C,” “MDG1C,” “Cummins,” “CM2350.”

File size heuristics. L5P calibration segments are typically 3.5-4.5 MB. CAN-am calibrations are typically 1.5-2.5 MB. If nothing else matches, the file size narrows the candidates.

Once the ECU family is identified, V-OP automatically fetches the matching A2L definition from cloud storage (previously uploaded definitions are stored to S3 and indexed by ECU family). The binary loads, the definition auto-fetches, the offsets auto-align, and the maps auto-populate. **Drop a file. See calibration data. That’s it.**

Erika — Your AI Calibration Professor

Erika is not a chatbot bolted onto a scan tool. She is a **calibration-aware AI advisor** that lives inside the editor and understands what you're looking at in real time.

What Erika Knows

Erika's knowledge base includes:

ECU architecture. She knows the difference between a GM E41 and a Bosch MG1CA920. She knows that the E41 uses a Tricore TC297 processor with MSB_LAST byte order, and that the MG1C uses a Tricore TC38x with MSB_FIRST. She knows the flash memory layout, the calibration segment boundaries, and the security access levels.

Calibration theory. She understands what every category of map does. Ask her "what does KaDFIR_FuelRailPressure do?" and she'll explain that it's the fuel rail pressure target table, indexed by RPM and fuel quantity, that the high-pressure fuel pump uses to maintain the commanded rail pressure. She'll tell you that increasing values in the high-load columns raises peak cylinder pressure and improves atomization, but going too high risks CP4 pump failure on certain model years.

Tuning relationships. She understands that boost targets, rail pressure targets, and injection timing are interdependent. If you increase boost without increasing rail pressure, you'll run lean. If you increase rail pressure without advancing timing, you're wasting fuel. She'll warn you about these relationships as you edit.

CAN bus protocols. She's trained on Vehicle Spy, SavvyCAN, CANape, CANoe, PCAN-View, and BusMaster concepts. She can explain arbitration IDs, DBC signal decoding, ISO-TP multi-frame assembly, UDS service identifiers, and negative response codes. When IntelliSpy captures an unknown CAN frame, Erika can help you figure out what it is.

Diagnostic reasoning. She's trained on the same fault detection logic that powers the diagnostic analyzer. She understands EGT spread patterns, rail pressure deviation signatures, boost leak indicators, and VGT actuator failure modes. She can look at your datalog and tell you not just what's wrong, but why it's wrong and what calibration changes might fix it.

Vehicle coding. She knows Ford IPC as-built data block encoding, RAM BCM configuration structures, and GM module addresses. She can walk you through a fuel tank size change or tire size correction step by step.

Powersports platforms. She knows CAN-am Rotax engine management, the MED17.8.5 vs MG1CA920 ECU timeline, the DESS key system, Sea-Doo iBR and supercharger boost control, Polaris dual-CAN architecture, and Kawasaki KDS diagnostics.

How Erika Works in the Editor

When you open Erika in the calibration editor, she receives context about your current session:

- Which A2L definition is loaded (ECU family, module info, map count)
- Which binary is loaded (file name, format, size, base address)
- The alignment result (offset, confidence, method)
- Which map you currently have selected (name, description, type, category, level, current values)
- Which maps you've modified and by how much

This means when you select a map and ask Erika "what does this do?", she doesn't give you a generic answer. She gives you a specific answer based on the actual map you're looking at, with context about the ECU platform and the values you're seeing.

Example interaction:

> ***You:*** *[Select KaBSTC_MaxBoostPressure in the editor]* ***You:*** *"What's a safe value for this at 3000 RPM?"*
>
> ***Erika:*** *"KaBSTC_MaxBoostPressure is the maximum allowable boost pressure target for the VGT control system on your L5P. At 3000 RPM, the stock value is typically 26.5 psi. For a stock-turbo L5P with supporting mods (intake, exhaust, intercooler), most tuners run 32-35 psi at that RPM point. Going above 38 psi on the stock turbo risks overspeeding the compressor wheel — you'll see compressor surge and the VGT will oscillate trying to maintain target. If you're running an S300 or S400 compound setup, 45-55 psi is typical at 3000 RPM depending on the charger sizing. What's your turbo configuration?"*

That’s not a generic answer from a manual. That’s a calibration-aware response that considers the ECU platform, the specific map, the RPM breakpoint, and real-world tuning experience.

Erika + IntelliSpy

When IntelliSpy is capturing CAN bus traffic, Erika can analyze the frame patterns:

- Identify which module is broadcasting based on arbitration ID ranges
- Decode known signals using the UDS DID reference database
- Spot anomalies — frames that appear at unexpected rates, data bytes that change when they shouldn’t, or missing modules that should be present
- Suggest DBC signal definitions for unknown frames based on byte change patterns

Erika + Diagnostics

When the diagnostic analyzer highlights a fault pattern in your datalog, Erika can:

- Explain the fault in plain language
- Show you which calibration maps are involved
- Suggest specific value changes that might address the issue
- Warn you about side effects of those changes

IntelliSpy — AI-Powered CAN Bus Sniffer

IntelliSpy is Vehicle Spy with a brain. It captures raw CAN frames from the PCAN bridge in real time and applies intelligence to every frame.

Real-time frame capture. The PCAN bridge runs a dedicated monitor loop that broadcasts every CAN frame to the browser via WebSocket. Frames arrive with microsecond timestamps, arbitration IDs, data length, and raw data bytes.

Three view modes:

- **Live:** Scrolling frame trace showing every frame as it arrives, with byte-level change highlighting (yellow for bytes that changed since the last frame with that arb ID)
- **Stats:** Per-arbitration-ID breakdown showing frame count, rate (Hz), last data, direction (Request/Response/Broadcast), and auto-identified module name
- **Decode:** AI-assisted module discovery that cross-references captured arb IDs against the 80-module database (48 Ford + 21 RAM + 11 GM modules)

Arb ID filtering. Click any ID in the sidebar to show/hide it. Filter to diagnostic-only traffic (0x700-0x7FF range). Search by ID, module name, or hex data pattern.

CSV export. Export all captured frames with timestamps for offline analysis in SavvyCAN, Vehicle Spy, or any other tool.

Vehicle Coding — Fuel Tank & Tire Size

The #1 requested feature from diesel truck owners: change the fuel tank size and tire size in the ECU without a dealer visit.

Ford diesel coding uses the IPC (Instrument Panel Cluster) as-built data system. The fuel tank capacity is encoded as a 12-bit value in block 720-01-01, bits 16-27. V-OP reads the current as-built block, decodes it (showing fuel tank size, dual sender configuration, flex fuel flag, transmission type, TPMS type, and eco cruise setting), lets you select a new tank size from 14 common options (23 gal to 80 gal), and writes the modified block back — automatically recalculating the checksum.

RAM diesel coding uses BCM configuration DIDs. V-OP reads the current configuration via UDS ReadDataByIdentifier ($22), presentsthedecodedvalues, andwriteschangesviaWriteDataByIdentifier$(2E) after security access.

Tire size correction includes a built-in tire size calculator. Enter your tire size in any format (35x12.50R20, 305/55R20, etc.) and V-OP calculates the diameter, circumference, and revolutions per mile. It then computes the exact speedometer error percentage compared to the factory tire size and writes the correction factor to the appropriate module.

Service Procedures — Guided UDS Wizards

Seven service procedures with step-by-step UDS command sequences:

1. **DPF Forced Regeneration** — Commands the ECU to perform a stationary DPF regen via RoutineControl ($31). Monitors EGT, soot load, and regen progress in real time.
2. **Injector Coding** — Writes IMA/IQA trim codes to the PCM after injector replacement. Supports GM, Ford, and RAM diesel formats.
3. **TPMS Sensor Relearn** — Programs new sensor IDs into the BCM. Supports OBD-triggered and manual relearn modes.
4. **Transmission Adaptive Reset** — Clears learned shift points and torque converter clutch apply pressures. Forces the TCM to re-learn from scratch.
5. **Oil Life / Service Reset** — Resets the oil life monitor and service interval counter via WriteDataByIdentifier.
6. **Throttle Body Alignment** — Runs the electronic throttle body learn procedure via RoutineControl.
7. **Steering Angle Calibration** — Calibrates the steering angle sensor after alignment work.

Each procedure is a guided wizard: connect → verify prerequisites → execute UDS sequence → verify success → report results. Error handling includes full NRC (Negative Response Code) decoding with human-readable explanations.

CAN-am VIN Changer

A dedicated wizard for changing the VIN on CAN-am powersports vehicles via the PEAK device:

1. **Connect** to the PCAN bridge
2. **Identify** the ECU (auto-detects MED17.8.5 vs MG1CA920)
3. **Read** the current VIN via ReadDataByIdentifier ($22 F190)
4. **Authenticate** via SecurityAccess ($27 level 3) with the appropriate seed/key algorithm

5. **Write** the new VIN via WriteDataByIdentifier ($2E F190)
6. **Reset** the ECU via ECUReset ($11 01)
7. **Guide** DESS key re-learn procedure

The tool handles both ECU variants and reports the exact NRC code if any step fails — so if a newer locked-down MG1CA920 rejects the security access, you see exactly which wall you hit and at which step.

The Diagnostic Analyzer — Fault Graphs with Reasoning

V-OP doesn't just read DTCs. It analyzes your datalog and builds **highlighted fault graphs** — visual charts where the problematic regions are marked with colored overlays and annotated with reasoning chains.

How it works:

1. Upload a CSV datalog (HP Tuners, EFILive, or V-OP's own datalogger format)
2. The diagnostic engine scans every PID channel for anomalies — values outside expected ranges, sudden spikes, correlation failures between related parameters
3. Each detected anomaly generates a **reasoning chain**: "Rail pressure dropped 400 bar below target at timestamp 12.4s while fuel command was 45mm^3/stroke → CP4 pump volume insufficient OR fuel filter restriction OR rail pressure relief valve leaking"
4. The fault graphs highlight the exact time window of each anomaly with color-coded overlays
5. A health report grades the vehicle across 8 systems (Fuel, Boost, Exhaust, Cooling, Transmission, Electrical, Emissions, Engine Mechanical) on a 0-100 scale

Drag timeslip analysis extracts 60-foot, 330-foot, 660-foot, and 1320-foot times from datalog acceleration events, calculates trap speed, and generates a formatted drag timeslip — complete with reaction time, ET, and MPH.

Dyno chart generation plots horsepower and torque curves from datalog data using calculated load, RPM, and acceleration. The charts include peak annotations, power band highlighting, and comparison overlays for before/after tune validation.

How Every Module Feeds Every Other Module

This is the architecture that no other tool has. Every module in V-OP is connected:

Datalogger → Diagnostic Analyzer: Raw PID data flows into the fault detection engine, which produces highlighted graphs and reasoning chains.

Diagnostic Analyzer → Erika: Fault reports and reasoning chains are injected into Erika's context, so she can explain what went wrong and suggest calibration fixes.

Erika → Calibration Editor: Erika's suggestions reference specific map names and value ranges, which the user can navigate to directly in the editor.

Calibration Editor → Binary Export: Modified map values are converted back to raw bytes and written to the binary at the correct offsets.

Binary Export → ECU → Datalogger: The modified binary is flashed to the ECU, the vehicle is driven, and new datalog data flows back into the analyzer — closing the loop.

IntelliSpy → UDS Reference → Erika: Captured CAN frames are cross-referenced against the module database and DID tables, with Erika providing real-time interpretation.

Module Scanner → Vehicle Coding → Service Procedures: The module scanner identifies which ECUs are present, vehicle coding reads/writes their configuration, and service procedures execute guided UDS sequences.

A2L Parser → Map Tree → Map Editor → Hex Editor: The A2L definition structures the map tree, the map tree navigates to the table editor, and the table editor links back to the hex editor for raw byte inspection.

Powersports PIDs → Datalogger → Diagnostic Analyzer: CAN-am, Sea-Doo, Polaris, and Kawasaki PIDs flow through the same datalogger and diagnostic pipeline as automotive PIDs.

The Numbers

Metric	Count
Lines of TypeScript	25,000+
Calibration map category patterns	80+
Tiered level assignment patterns	40+
Search ranking tiers	10
Binary format readers	4 (Raw, S-Record, Intel HEX, PPEI Container)
ECU family detectors	5 (filename, magic bytes, embedded strings, size heuristics, A2L content)
Offset alignment strategies	3 (base address, known offsets, brute force)
Ford module addresses	48
RAM module addresses	21
GM module addresses	11
Powersports PIDs	94 (CAN-am 28, Sea-Doo 22, Polaris 26, Kawasaki 18)
Service procedures	7
UDS NRC codes decoded	30+
Seed/key algorithms	4 (GM, CAN-am, Cummins, Ford)
Vitest test cases	534
Days since first line of code	Less than 7

What’s Coming Next

IntelliSpy Session Recording. Capture full CAN bus sessions and replay them with decode overlay for offline reverse engineering.

Output Control / Actuator Testing. Bidirectional IO control ($2F) — command individual solenoids, injectors, relays, and actuators through the PCAN bridge. Fire injector #3. Cycle the EGR valve. Command the VGT to a specific position. Test anything.

As-Built Data Community Database. FORScan-style community spreadsheets decoded into V-OP's coding engine, so every bit in every as-built block has a human-readable description.

Real-Time Erika + IntelliSpy. Erika watches the CAN bus in real time and provides running commentary on what she sees — identifying modules, decoding signals, spotting anomalies, and suggesting investigations.

ECU Flash Read/Write. When the security research catches up, V-OP's UDS transport layer is already built to support RequestDownload $(34), TransferData(36)$, and RequestTransferExit ($37). The protocol is ready. The security access is the only wall.

> *Built in less than a week. Already deeper than tools that have been on the market for a decade. And every day, I learn more. The code reader is dead. Welcome to V-OP.*

V-OP by PPEI — Vehicle Optimizer *Where AI meets the engine bay.*